Mestrado em Engenharia Informática
Gestão de Identidade

Semana 4 — MFA com Authenticator App (TOTP)

Ricardo Costa
rcosta@estg.ipp.pt
Escola Superior de Tecnologia e Gestão do Instituto Politécnico do Porto

Objetivos da Semana 4

No final da semana os estudantes deverão conseguir:

- Explicar o papel do MFA na redução de risco
- Distinguir TOTP de outros fatores (SMS, Push, FIDO2)
- Implementar setup e validação de TOTP
- Integrar MFA no fluxo de login
- Compreender recovery codes e boas práticas

Agenda (4h)

- **Teoria (1h30)
  - Conceitos de MFA
  - TOTP: como funciona
  - Riscos e controlos
- **Prática (2h45)**
  - Setup de MFA com app authenticator
  - Verificação de código 6 dígitos
  - Recovery codes e testes

Porque MFA?

Password sozinha é frágil:

- reutilização de passwords
- phishing
- credential stuffing

MFA adiciona um fator extra e reduz risco de compromisso de conta.

Tipos de fator

- **Know**: algo que sabes (password/PIN)
- **Have**: algo que tens (telemóvel/token)
- **Are**: algo que és (biometria)

MFA = combinar fatores de categorias diferentes.

TOTP em linguagem simples

TOTP (*Time-based One-Time Password*):

- código curto (normalmente 6 dígitos)
- muda de 30 em 30 segundos
- gerado na app authenticator e no servidor a partir do mesmo secret

Fluxo MFA com TOTP

1. Utilizador faz login com password
2. Sistema pede código TOTP
3. Utilizador abre app authenticator e introduz código
4. Sistema valida
5. Acesso concedido

Como o TOTP é calculado (conceito)

Base técnica (resumo):

- parte de um `secret` partilhado entre servidor e app
- combina esse secret com o tempo atual (time step, ex.: 30s)
- aplica HMAC (normalmente SHA-1/256/512)
- produz código curto (ex.: 6 dígitos)

Consequência prática:

- se relógios estiverem desalinhados, validação falha

Parâmetros que importam no TOTP

- **time step**: 30s (mais comum)
- **digits**: 6 (mais comum)
- **window**: tolerância de validação (ex.: -1/+1 step)

Trade-off:

- janela maior -> melhor UX
- janela maior -> mais superfície para brute force/replay

Threat model prático de MFA

MFA reduz risco de:

- password reuse
- credential stuffing
- brute force de password

MFA não elimina totalmente:

- phishing em tempo real
- malware no endpoint
- account recovery fraco

Enrollment, reset e desativação (governance)

Processos críticos além do login:

- **enrollment**: ativação inicial segura
- **reset**: quando utilizador perde telemóvel
- **disable**: quem pode desativar MFA e com que prova

Se estes fluxos forem fracos, MFA perde valor.

MFA adaptativo (conceito)

Nem sempre precisamos desafiar da mesma forma.

Exemplos de sinal de risco:

- novo dispositivo
- geolocalização anómala
- horário invulgar
- recurso sensível

Decisão típica:

- baixo risco: sessão normal
- alto risco: pedir MFA adicional ou bloquear

Setup inicial do MFA

No primeiro setup:

- servidor gera `secret`
- app apresenta QR code (otpauth URL)
- utilizador adiciona conta na authenticator app
- sistema pede 1 código para confirmar ativação

TOTP vs SMS vs Push vs FIDO2

- **TOTP**: simples, offline, bom para ensino/lab
- **SMS**: fraco contra SIM swap
- **Push**: melhor UX, exige serviço push
- **FIDO2/WebAuthn**: mais forte contra phishing

Riscos ainda existentes

MFA ajuda, mas não resolve tudo:

- phishing em tempo real (adversary-in-the-middle)
- roubo do seed TOTP
- engenharia social no reset de conta

Boas práticas MFA

- proteger secrets em storage seguro
- limitar tentativas de código
- clocks sincronizados (NTP)
- permitir recovery codes de uso único
- auditar ativações/desativações de MFA

Resultado esperado da prática

Cada grupo deve demonstrar:

- setup MFA com QR code
- verificação TOTP com sucesso
- falha com código inválido/expirado
- uso de recovery code

Transição para prática

Vamos implementar numa app mínima:

- login base
- página de setup MFA
- página de verificação MFA
- sessão autenticada só após MFA válido

Prática — pré-requisitos

- Node.js 18+
- App Authenticator no telemóvel (Google / Microsoft / Authy)

Instalar dependências:

```
mkdir gi-semana4-mfa
cd gi-semana4-mfa
npm init -y
npm install express express-session speakeasy qrcode dotenv
```

Prática — ficheiro .env

```
PORT=3000
SESSION_SECRET=trocar-por-um-segredo-forte
```

Prática — código base da app

Usar ficheiro disponibilizado:

- `semana4-app-base.js`

Guardar como `app.js`.

Exercício extra (Semana 4) — MFA TOTP no Login Google (Semana 3)

Objetivo: pegar na app da **Semana 3 (Login with Google / OIDC)** e adicionar **MFA via TOTP**.

Ideia-chave: depois do OIDC login, o utilizador só acede a zonas protegidas após confirmar MFA.

Exercício extra — Dependências

No projeto da Semana 3, instalar:

```
npm install speakeasy qrcode
```

Exercício extra — Código base

Substituir `app.js` pelo ficheiro base:

- `semana4-app-base-google-mfa.js`

> Mantém o `.env` da Semana 3 (não há novas variáveis obrigatórias).

Exercício extra — Fluxo de teste

1. Login com Google
2. Setup MFA (QR)
3. Confirmar TOTP
4. Logout/login
5. Validar MFA e aceder a:
    - `/profile`
    - `/secure`

Exercício extra — Checklist

- [ ] Login OIDC funciona
- [ ] Setup MFA via QR code
- [ ] TOTP válido aceite
- [ ] TOTP inválido rejeitado
- [ ] `/profile` protegido por MFA
- [ ] `/secure` protegido por MFA

Prática — executar e testar

```
node app.js
```

Fluxo de teste:

1. login com utilizador demo
2. setup MFA (QR)
3. validar código da app
4. logout/login e revalidar MFA

Prática — troubleshooting

- código sempre inválido -> hora do sistema errada
- QR não lê -> secret mal formatado
- loop de login -> estado de sessão mal gerido

Mini-quiz rápido

1. O que é TOTP?
2. Porque MFA reduz risco de credential stuffing?
3. Qual limitação importante de SMS OTP?
4. Para que servem recovery codes?

Respostas mini-quiz

1. Código temporário baseado em tempo e secret partilhado
2. Password roubada deixa de ser suficiente
3. Vulnerável a SIM swap
4. Recuperar acesso quando o segundo fator falha/perde-se

Fecho

Mensagem-chave:

> MFA com TOTP é um passo forte e prático para elevar segurança de autenticação em sistemas reais.